

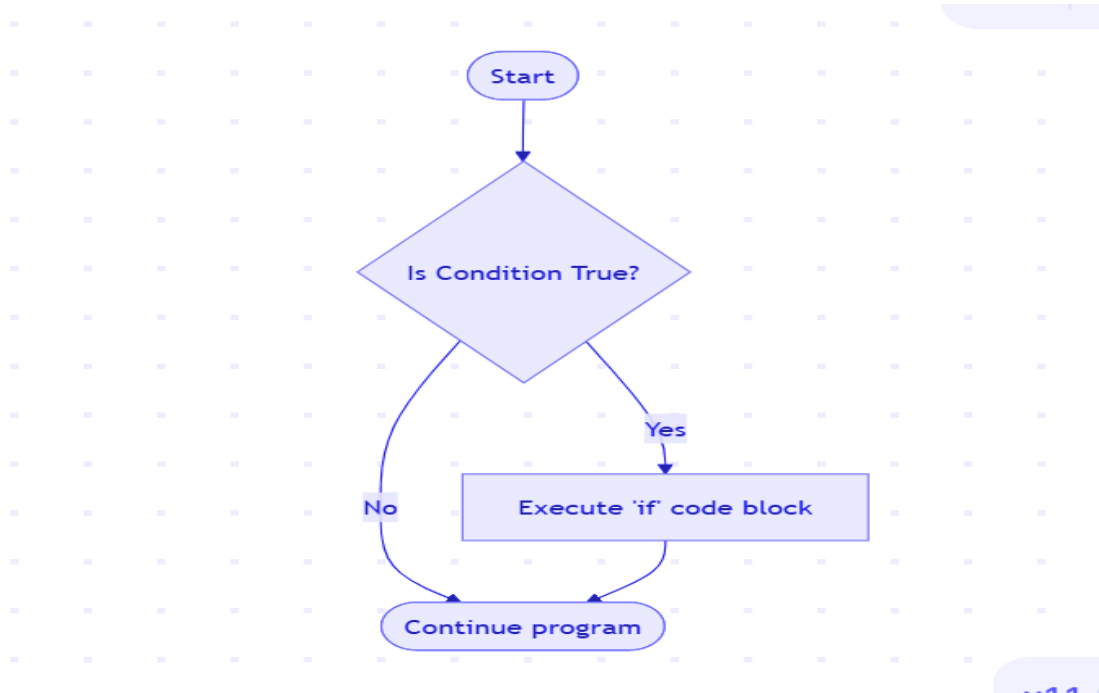
Conditional Statements and Loops

1. Conditional Statements

- Conditional statements control the flow of execution in a program.
- They allow different blocks of code to run depending on whether a specific condition evaluates to True or False.

1.1 The if Statement

Function: Executes a block of code only when a specified condition evaluates to True.



Syntax

```
Python
if condition:
    # Code to execute
```

Example

Python

```
age = 20
if age >= 18:
    print("You are an adult.")
```

Output

```
You are an adult.
```

1.2 Short Hand if

Function: Used to write an if statement on a single line.

- This is useful when only one statement needs to be executed.

Syntax

Python

```
if condition: statement
```

Example

Python

```
if 10 > 5: print("10 is greater than 5")
```

Output

```
10 is greater than 5
```

1.3 The if-else Statement

Function: Executes one if block code when the condition is True and else block code when the condition is False.

Syntax

```
Python
if condition:
    # Code if True
else:
    # Code if False
```

Example:

```
Python
temperature = 30
if temperature > 25:
    print("It's a hot day.")
else:
    print("It's a cool day.")
```

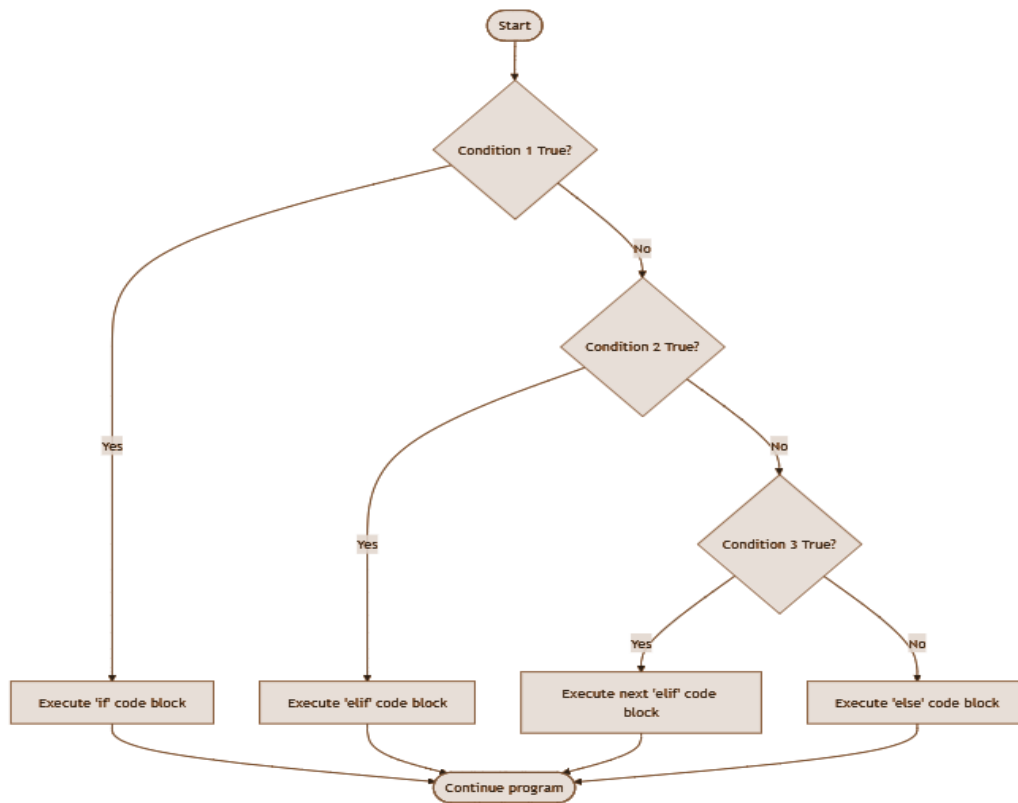
Output:

```
It's a hot day.
```

1.4 The if-elif-else Statement

Function: The elif statement allows you to check multiple conditions in order.

- It executes a block of code when its condition evaluates to True after previous conditions have evaluated to False.



Syntax:

Python

```
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition2 is True
else:
    # Code if all previous are False
```

Example:

Python

```
score = 85
```

```
if score >= 90:  
    print("Grade: A")  
elif score >= 80:  
    print("Grade: B")  
else:  
    print("Grade: C or below")
```

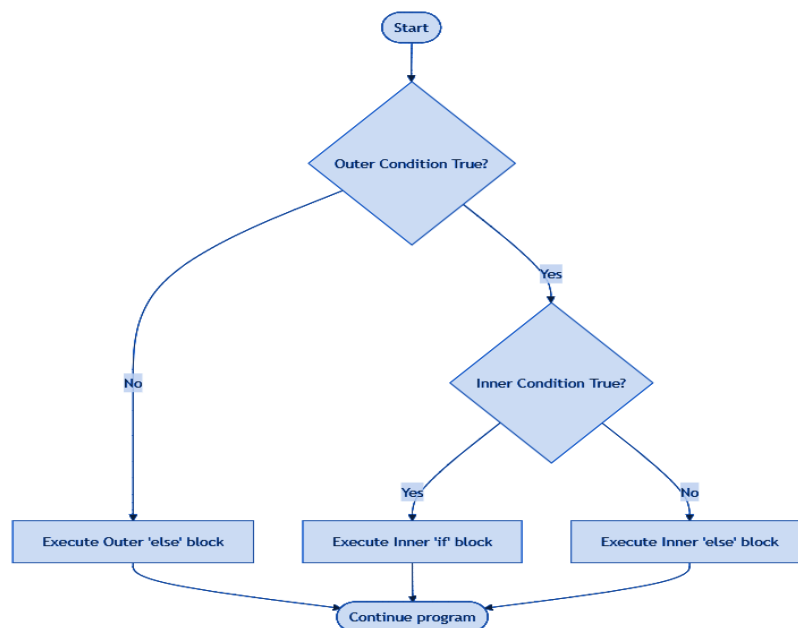
Output:

Grade: B

1.5 Nested if-else Statement

Function: An if or else block placed inside another if or else block.

- It is used to check conditions within another condition.



Syntax

```
Python
if condition1:
    if condition2:
        # Code if both are True
```

Example

```
Python
num = 15
if num > 0:
    if num % 2 == 0:
        print("Positive even number")
    else:
        print("Positive odd number")
```

Output:

```
Positive odd number
```

1.6 Conditional Expression (Ternary Operator)

Function: A shorthand way to write an if-else statement on a single line, generally used when assigning one of two values based on a condition.

- The else part is strictly required.

Syntax

Python

```
variable = value_if_true if condition else value_if_false
```

Example

Python

```
age = 16
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output

Minor

1.7 Match-Case Statement

Function: Introduced in modern Python, it compares a value against multiple patterns and executes the matching block of code, similar to a switch-case statement in other languages.

- The underscore `_` acts as a default statement when none of the cases are satisfied.

Syntax

Python

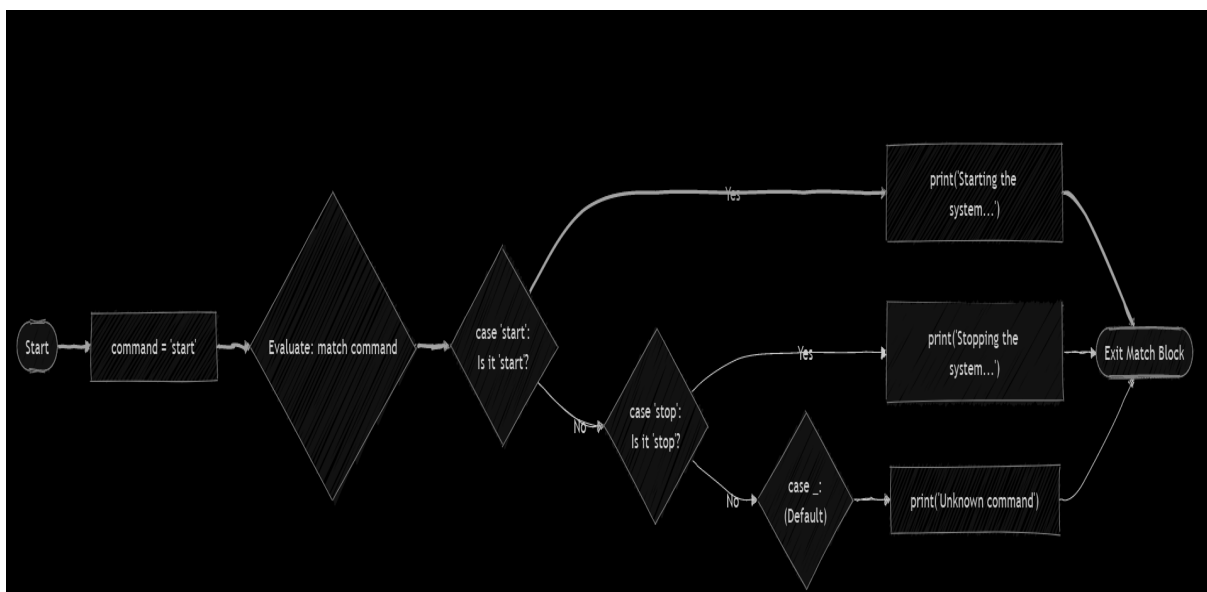
```
match subject:
    case pattern1:
        # Code block
    case _:
        # Default code
```

Example

Python

```
command = "start"

match command:
    case "start":
        print("Starting the system...")
    case "stop":
        print("Stopping the system...")
    case _:
        print("Unknown command")
```



Output

```
Starting the system...
```

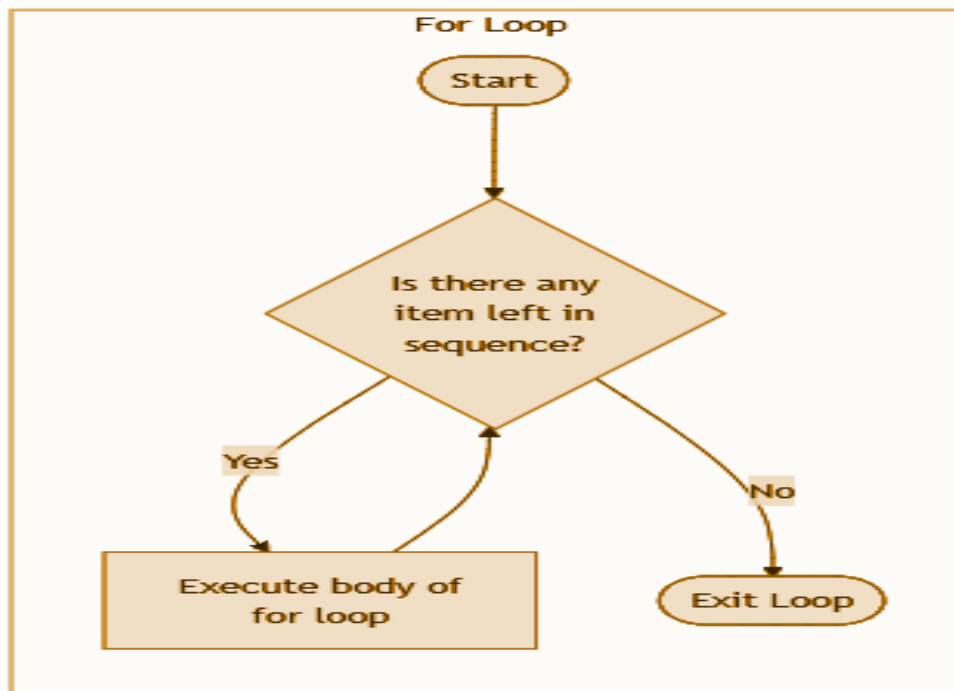
2. Loops in Python

- Loops execute a block of code repeatedly until a condition is met or all items in a sequence are processed.

2.1 The for Loop

Function: Used to iterate over a sequence (such as a list, tuple, string, or range()).

- It executes a block of code once for each item in the sequence.



Syntax

Python

```
for item in sequence:  
    # Code to execute
```

Example

Python

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)
```

Output

```
apple  
banana  
Cherry
```

2.2 Iterating by Index

Function: You can use a for loop to iterate through sequence elements using their index values by combining the range() and len() functions.

Syntax

```
Python  
for index in range(len(sequence)):  
    # Code to execute
```

Example

```
Python  
a = ["Python", "Java", "C++"]  
for i in range(len(a)):  
    print(f"Index {i}: {a[i]}")
```

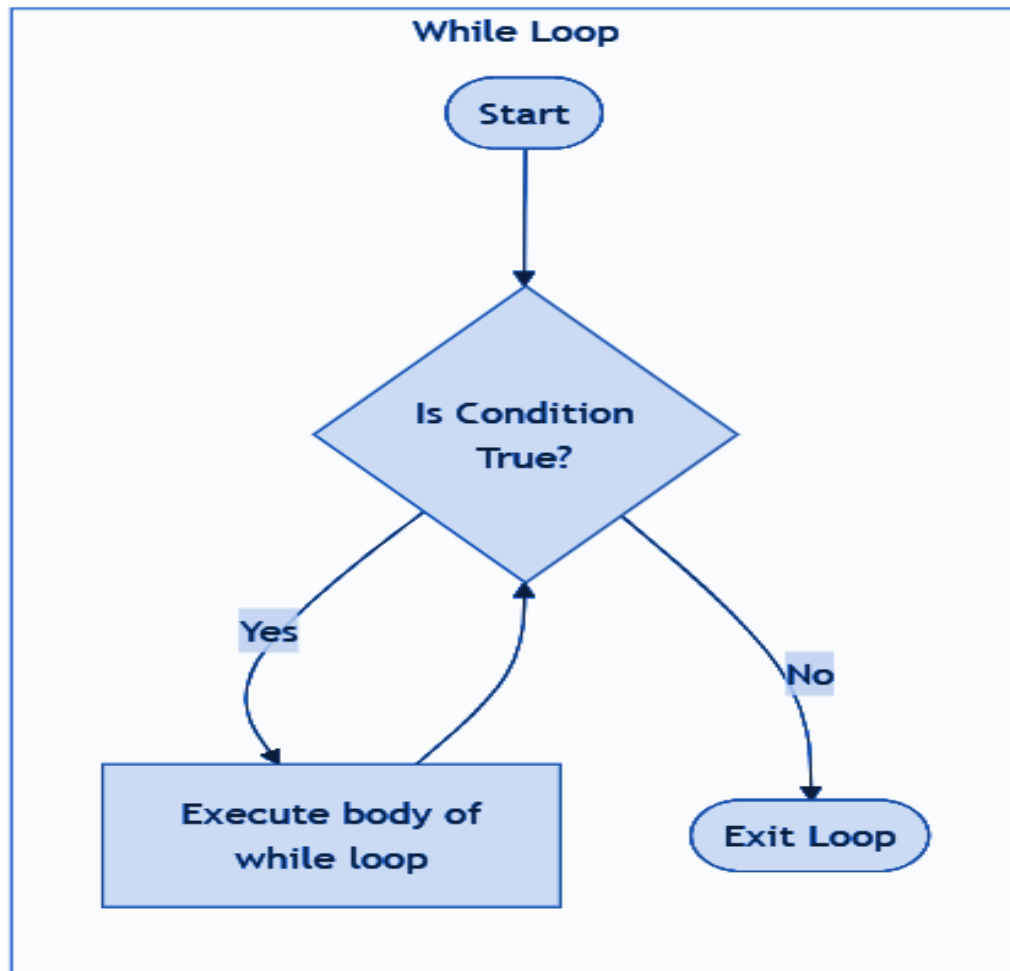
Output

```
Index 0: Python  
Index 1: Java  
Index 2: C++
```

2.3 The while Loop

Function: Repeatedly executes a block of code as long as the given condition remains True.

- When the condition becomes False, the line immediately after the loop is executed.



Syntax

```
Python
while condition:
    # Code to execute
```

Example

Python

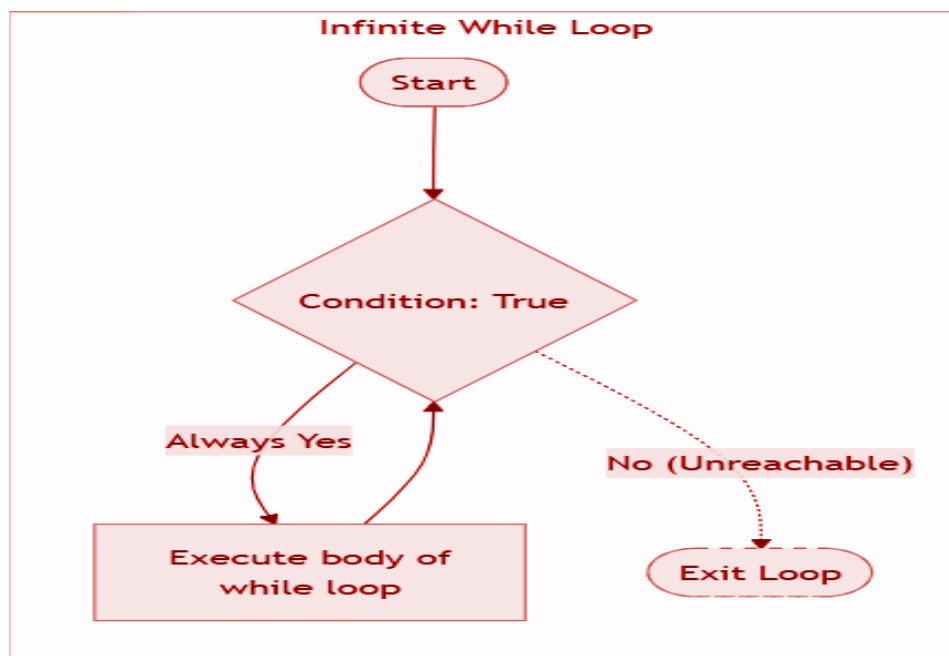
```
cnt = 0
while cnt < 3:
    print(f"Counter is: {cnt}")
    cnt += 1
```

Output

```
Counter is: 0
Counter is: 1
Counter is: 2
```

2.4 Infinite while Loop

Function: If a loop's condition always evaluates to True, it runs continuously in a never-ending infinite loop, requiring you to forcefully terminate the program.



Syntax

Python

```
while True:  
    # Code to execute forever
```

Example

Python

```
# Warning: This will print endlessly until forcefully stopped  
while True:  
    print("Stuck in a loop!")
```

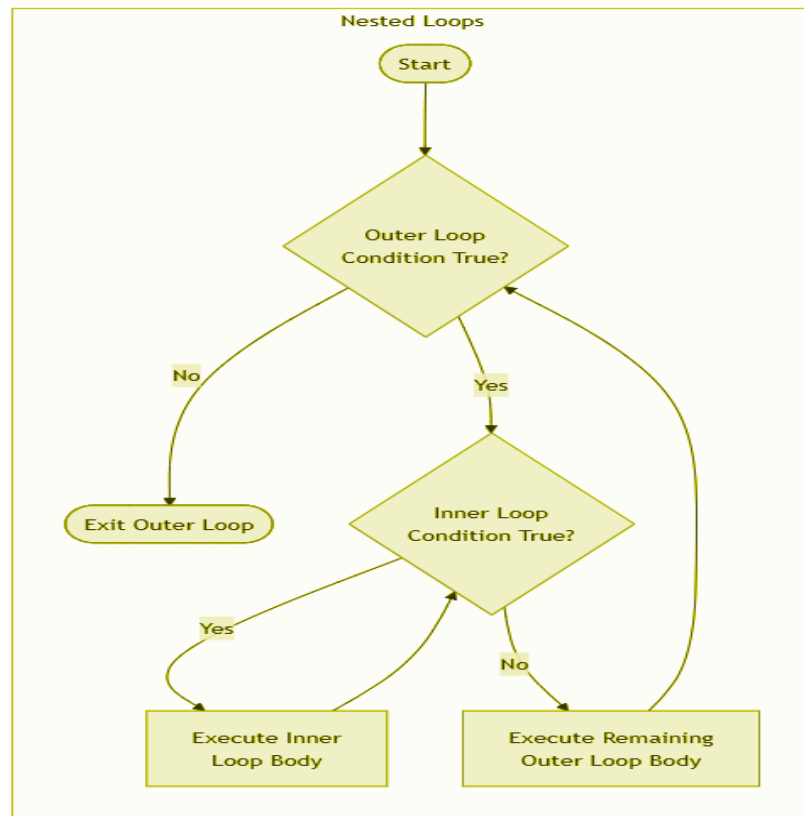
Output

```
Stuck in a loop!  
Stuck in a loop!  
Stuck in a loop!  
...
```

2.5 Nested Loops

Function: A loop inside another loop.

- The inner loop executes completely for every single iteration of the outer loop.



Syntax

Python

```
for outer_item in outer_sequence:
    for inner_item in inner_sequence:
        # Code to execute
```

Example

Python

```
for x in range(1, 4):
    for y in range(1, 4):
        print(f"({x},{y})", end=" ")
    print()
```

Output

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

2.6 Loop Control

2.6.1 break statement

Function: The break statement breaks out of the innermost enclosing for or while loop.

Syntax

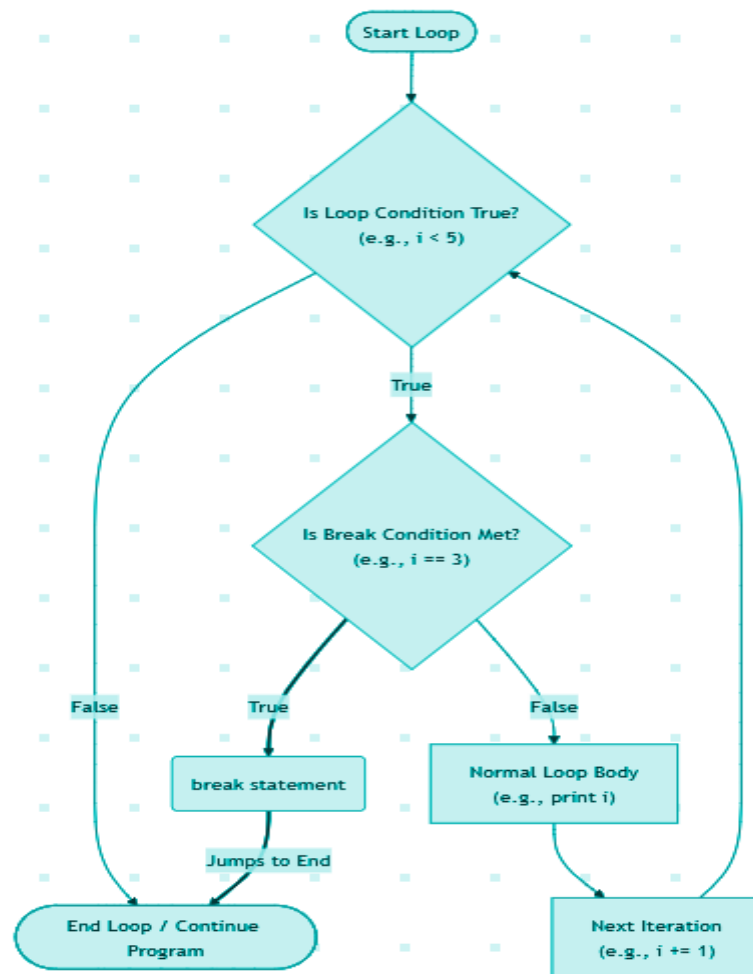
Python

```
# Placed within an if condition inside the loop
if condition:
    break
```

Example

Python

```
for i in range(5):
    if i == 3:
        print("Breaking out!")
        break
    print(i)
print("Loop finished.")
```



Output

```
0
1
2
Breaking out!
Loop finished.
```

2.6.2 The continue Statement

Function: Skips the rest of the current loop iteration and immediately jumps to the next iteration.

Syntax

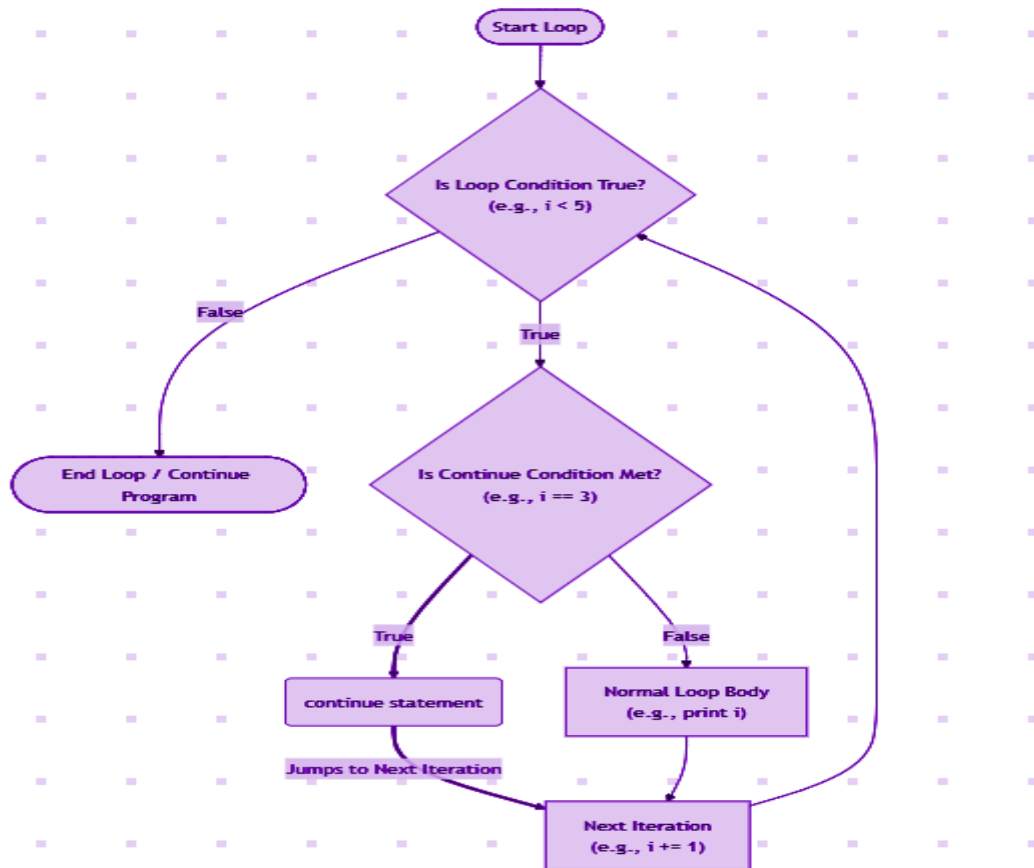
Python

```
if condition:  
    continue
```

Example

Python

```
for i in range(5):  
    if i == 3:  
        print("Skipping 3!")  
        continue  
    print(i)  
print("Loop finished.")
```



Output

```
0
1
2
Skipping 3!
4
Loop finished.
```